

Transformer 不是逐字“读完再记住”，而是把一串 token 同时变成向量，然后让每个位置去询问其他位置：**哪些 token 对理解我最有用？**

- **token / embedding**：文本先被 tokenizer 切成 token；每个 token ID 查表变成 embedding 向量，再加入位置信息。
- **Q/K/V**：每个 token 的向量会投影成三种角色：Q 是“我在找什么”，K 是“我能被什么匹配”，V 是“如果被关注，我贡献什么内容”。
- **attention score**：用 $QK^T / \sqrt{d_k}$ 计算 token 之间的相关性分数。
- **softmax**：把分数变成概率权重，权重越大，说明当前位置越关注那个 token。
- **multi-head**：不是只看一种关系，而是多组 Q/K/V 并行看不同关系；例如一头看主语，一头看指代，一头看语义搭配。
- **residual**：把子层输出加回原输入，避免信息在深层网络中丢失，也让训练更稳定。
- **layer norm**：归一化每个位置的表示，稳定数值分布。原始 Transformer 是 子层 -> residual -> layer norm；很多现代 LLM 常用 Pre-LN，即先 norm 再进 attention/MLP。
- **MLP / FFN**：attention 负责“从上下文取信息”，MLP 负责“对每个位置独立做非线性变换”。

Encoder-Decoder vs Decoder-Only LLM

原始 Transformer 是 **encoder-decoder** 架构：encoder 读取完整输入序列，decoder 一边看已生成的目标 token，一边通过 cross-attention 读取 encoder 输出。适合机器翻译这类“输入序列 -> 输出序列”的任务。

现代 GPT 类 LLM 多是 **decoder-only**：只有 decoder 堆叠，没有独立 encoder，也通常没有 cross-attention。它使用 **causal mask**，每个 token 只能看自己和之前的 token，训练目标是预测下一个 token。因此它读取上下文的方式是：把 prompt 作为已知前文，在每一层 masked self-attention 中不断重写每个位置的上下文表示。

附、怎么理解多组 Q/K/V 并行看不同关系？

输入一个 X，在同一个 Transformer block 里会生成**多组 Q/K/V**。

更准确地说，X 通常不是单个 token，而是整段序列的表示矩阵：

X: [序列长度, d_{model}]

如果有 h 个 attention head，那么第 i 个 head 会有自己的一套参数：

$Q_i = X W_{Q_i}$ $K_i = X W_{K_i}$ $V_i = X W_{V_i}$

所以同一个 X 会被投影成多组：

Head 1: Q_1, K_1, V_1
Head 2: Q_2, K_2, V_2
Head 3: Q_3, K_3, V_3 ... Head h: Q_h, K_h, V_h

每一组都会独立算一次 attention:

$$\text{head}_i = \text{softmax}(Q_i K_i^T / \sqrt{d_k}) V_i$$

最后把所有 head 的结果拼起来:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W_O$$

关键点是：这些 head 看的是**同一批 token**，但因为 $W_{Q_i} / W_{K_i} / W_{V_i}$ 不同，所以它们把 token 投影到了不同的表示空间里。

所以“看不同关系”不是人为规定的，比如“第 1 个头看主语，第 2 个头看宾语”。而是训练过程中，不同 head 可能自然学到不同的关注模式。

例如句子：

小明 把 书 放在 桌上，因为 它 很厚。

当模型处理“它”时：

- 某个 head 可能强烈关注“书”，解决指代关系；
- 某个 head 可能关注“很厚”，理解属性；
- 某个 head 可能关注附近 token，捕捉局部语法；
- 某个 head 可能关注“放在 桌上”，理解事件结构。

但这些都是学习出来的倾向，不是硬编码规则。

可以把 multi-head 理解成：

同一个上下文，被多个不同的“阅读视角”同时阅读，然后把这些视角的结果合并。